ResolveAI — System Design

Oct 5, 2026 · @Pipe Gen

1. Goals and scope

ResolveAI is a multi-tenant SaaS that answers customer questions from a business's own documents, triages support tickets, calls approved tools such as order lookup, and drafts replies that a human approves before anything consequential happens. The design is a **modular monolith**: one FastAPI codebase deployed as two processes (API and worker), one Postgres database, and Redis. It stays simple to build in 6 weeks and can be split into services later without rewriting the domain code.

In scope for the MVP

- Business admin signs up, creates a workspace, invites agents.
- Admin uploads documents (PDF, DOCX, TXT, MD) to a knowledge base.
- Customers chat through an embeddable widget and get answers with source references, or are escalated to a ticket.
- Incoming messages are classified (intent, category, priority) with schema-validated output.
- The agent can call a mock order-status API and propose actions such as refunds.
- Every outbound ticket reply and every sensitive action needs human approval.
- Dashboard shows tickets, conversations, feedback, usage and cost.

Not in the MVP: real help-desk or e-commerce integrations, email inbox ingestion, billing, multilingual and voice, MCP server, mobile app, hybrid retrieval with reranking. The design leaves a seam for each.

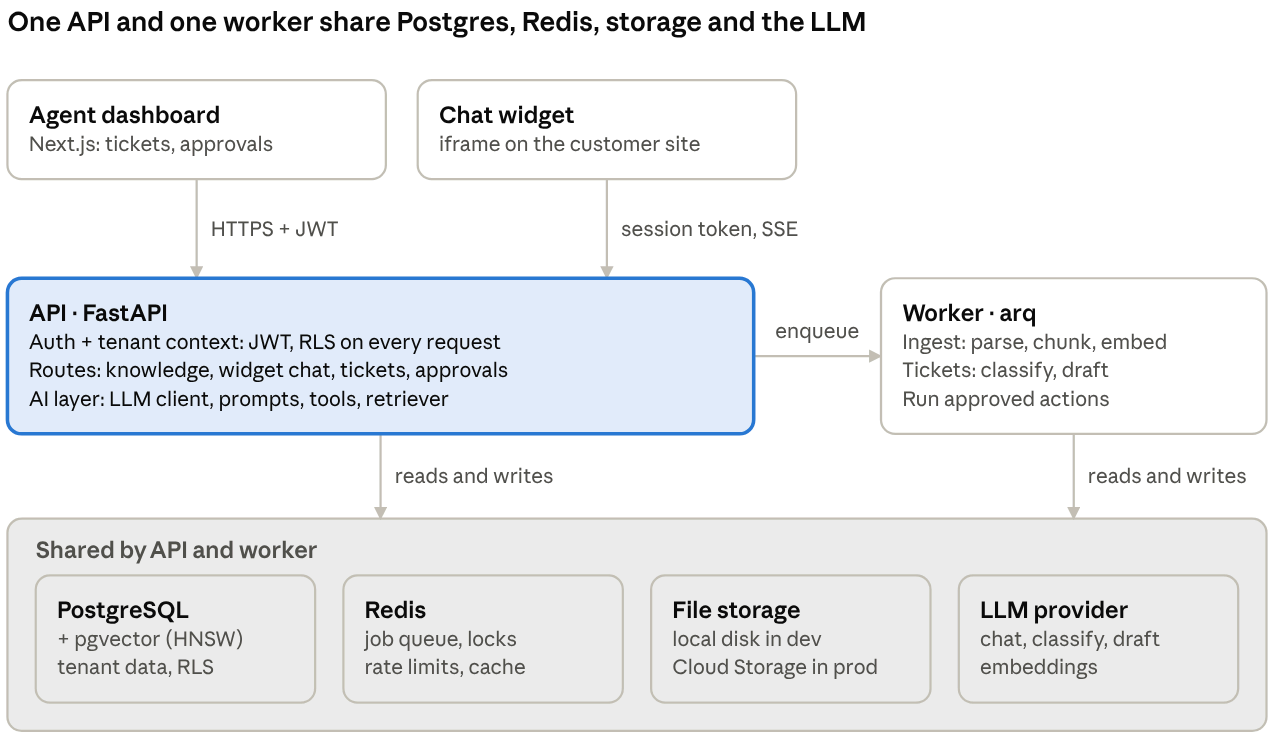
Quality targets

Attribute	Target
Tenant isolation	Zero cross-workspace reads; enforced in the app layer and by Postgres row-level security
Chat answer latency	First token under 2 s, full answer under 8 s (p95)
Ingestion	A 50-page PDF is searchable within 2 minutes

Attribute	Target
Safety	No side-effecting tool runs without an approved action record
Reliability	Every job retries with backoff; every action is idempotent
Cost visibility	Tokens and cost recorded for every LLM call, per workspace

2. Architecture overview

Two clients talk to one API; the API hands slow work to one worker through Redis; both processes share Postgres, Redis, file storage and the LLM provider.



System architecture · 2 clients, 2 processes, 4 shared dependencies

The API answers fast and never waits on ingestion or drafting; anything slow becomes a job the worker picks up.

Key decisions

- **Modular monolith.** One Python codebase with feature modules (`auth`, `workspaces`, `knowledge`, `conversations`, `tickets`, `approvals`), run as two processes. Splitting a module into its own service later is a deployment change, not a rewrite.
- **Postgres is the single source of truth**, including vectors. No separate vector database to keep in sync, and RLS covers embeddings too.
- **Every model call goes through one AI layer**, which owns prompts, tools, retries, cost and guardrails.
- **Side effects only happen through approved action records.** The model proposes; a human approves; the worker executes once.
- **Routers stay thin.** `app/api/` handles HTTP; `app/modules/<feature>/service.py` holds the logic, so the worker and API share it.

3. Tech stack

The stack follows the project overview, with two deliberate choices: **arq** instead of Celery for jobs, because it is asyncio-native and shares the async SQLAlchemy code, and a hand-written agent loop instead of LangGraph until a real need appears.

Layer	Choice	Why
Web app	Next.js 15 (App Router), TypeScript, Tailwind, shadcn/ui, TanStack Query	Dashboard and agent console
Chat widget	Small React bundle served by the web app, embedded via a script tag	Customer-facing chat on any site
API	Python 3.12, FastAPI, Pydantic v2	Typed request/response, OpenAPI for free
ORM and migrations	SQLAlchemy 2.0 async (asyncpg), Alembic	One model layer for API and worker
Database	PostgreSQL 16 + pgvector (HNSW index)	Relational data and vectors in one place, row-level security
Cache, rate limits, queue	Redis 7	One dependency for three jobs

Layer	Choice	Why
Background jobs	arq (Redis-backed, async)	Ingestion, embedding, classification, sending
File storage	Local disk in dev, Google Cloud Storage in prod, behind one interface	Uploaded documents
LLM	One provider behind an `LLMClient` interface	Swap providers without touching features
Embeddings	One embedding model, dimension fixed in config	Must stay the same for a workspace's whole index
Document parsing	pypdf, python-docx, plain text/Markdown	Covers the MVP file types
Auth	Own JWT (access + rotating refresh), argon2 password hashing	Full control, good learning value
Observability	structlog JSON logs, OpenTelemetry traces, Sentry for errors	Request IDs tie logs, traces and LLM calls together
Tests and evals	pytest, httpx, a custom eval runner in `evals/`	Unit, API and AI-quality tests
Deploy	Docker, GitHub Actions, Cloud Run, Cloud SQL, Memorystore, Secret Manager	Managed, scales to zero for the API

4. Multi-tenancy, auth and security

Every row of business data carries a `workspace_id`, and isolation is enforced twice: the application scopes every query, and Postgres row-level security (RLS) blocks anything the application misses.

Tenancy

- Shared database, shared schema, `workspace_id` on every tenant table (cheapest model that still isolates well).

- The workspace is always derived from the authenticated principal, never from a request body or query string.
- Each request opens a transaction and runs `SET LOCAL app.workspace_id = '<id>'`. RLS policies on every tenant table allow only rows where `workspace_id = current_setting('app.workspace_id')::uuid`.
- The API connects as a non-owner role (`resolveai_app`) so RLS cannot be bypassed; migrations run as the owner role.
- A repository base class adds the `workspace_id` filter automatically; a test suite tries cross-tenant reads on every endpoint and must get 404.

Principals and roles

Principal	How it authenticates	Can do
Owner	Email + password, JWT	Everything, including billing and deleting the workspace
Admin	Email + password, JWT	Manage members, knowledge base, settings, approve anything
Agent	Email + password, JWT	Work tickets, edit and approve drafts, approve low-risk actions
Customer (widget)	Workspace public key + short-lived session token	Chat in their own conversation only
Worker	Internal; carries `workspace_id` in each job payload	Runs jobs under that workspace's RLS context

Auth details

- Access token: JWT, 15 minutes, contains `sub` (user id), `wid` (active workspace), `role`.
- Refresh token: opaque, 30 days, stored hashed, rotated on every use; reuse of an old token revokes the whole family.
- Web app keeps the refresh token in an httpOnly, Secure, SameSite=Lax cookie; the access token lives in memory.
- Widget: the page embeds a public workspace key; the API exchanges it for a 1-hour customer session token bound to one conversation and to the site's allowed origins.

Other safeguards

- Rate limits in Redis (token bucket): per IP and per workspace on the widget, per user on the API, plus a daily LLM token budget per workspace.
- Uploads: 20 MB limit, MIME sniffing, extension allowlist, stored under random keys, never served back raw without an auth check.
- Secrets only in environment variables or Secret Manager; logs redact emails, phone numbers and tokens.
- CORS: the dashboard origin for the API; per-workspace allowed origins for widget endpoints.
- Audit log for every login, role change, approval, rejection and executed action.

5. Data model

Sixteen tables cover the MVP. All use UUID primary keys and timezone-aware `created_at` / `updated_at`; every table except `users` and `refresh_tokens` has `workspace_id` and an RLS policy. Status columns are Postgres enums or checked text, never free text.

Table	Key columns	Notes
`workspaces`	name, slug (unique), widget_public_key, allowed_origins[], settings jsonb	Settings hold AI options: tone, auto-answer on/off, confidence threshold
`users`	email (unique), password_hash, full_name, is_active	Global; a user can belong to many workspaces
`memberships`	user_id, workspace_id, role (owner/admin/agent)	Unique (user_id, workspace_id)
`refresh_tokens`	user_id, token_hash, family_id, expires_at, revoked_at	Rotation and reuse detection
`contacts`	email, name, external_id	The business's customers

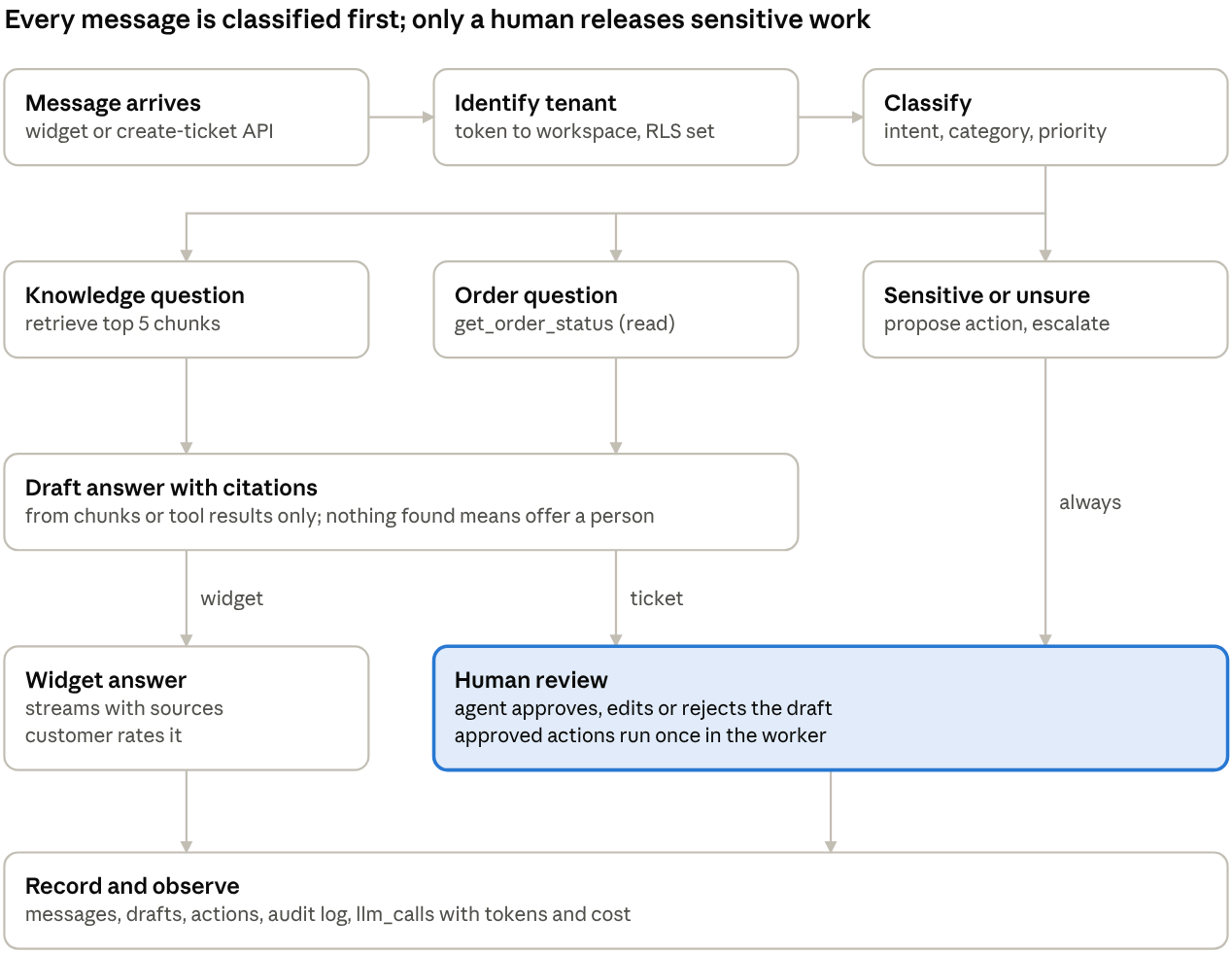
Table	Key columns	Notes
`documents`	title, mime_type, storage_key, content_hash, status (uploaded/processing/ready/failed), error, chunk_count, uploaded_by	Unique (workspace_id, content_hash) stops duplicate ingestion
`document_chunks`	document_id, chunk_index, content, token_count, embedding vector(N), tsv tsvector, metadata jsonb	HNSW index on embedding (cosine), GIN on tsv for later hybrid search
`conversations`	contact_id, channel (widget/email/api), status (open/escalated/closed), ticket_id	One thread of messages
`messages`	conversation_id, role (customer/assistant/agent/system), content, citations jsonb, llm_call_id	Citations = chunk ids + document titles
`tickets`	conversation_id, subject, intent, category, priority (low/normal/high/urgent), status, assignee_id, classification jsonb, confidence	Status machine in section 6
`reply_drafts`	ticket_id, content, citations, status (pending/approved/edited/rejected/sent), reviewed_by, reviewed_at, final_content	The AI's proposed reply
`action_requests`	ticket_id, tool_name, arguments jsonb, risk (read/write/sensitive), status, idempotency_key (unique), result jsonb, approved_by	Every side-effecting tool call
`mock_orders`	order_number, contact_email, status, carrier, tracking_number, eta, total, items jsonb	Seeded per workspace; stands in for a real shop
`feedback`	message_id or draft_id, source (customer/agent), rating (up/down), comment	Feeds evals and the dashboard
`llm_calls`	purpose, provider, model, prompt_version, input_tokens, output_tokens, cost_usd, latency_ms, status, trace_id	One row per LLM or embedding call

Table	Key columns	Notes
`audit_logs`	actor_type, actor_id, action, entity_type, entity_id, data jsonb, ip	Append-only; no updates or deletes

The embedding dimension N is set once from config (for example 1024 or 1536) and fixed by migration. Changing the embedding model later means a new column or table plus a re-embed job, not an in-place change.

6. Core flows

Four flows carry the product: document ingestion, live chat answers, the ticket pipeline, and approved actions. Anything slow or retryable runs in the worker; the API only validates, writes a record, and enqueues.



End-to-end request flow · 3 routes, 1 human checkpoint

Widget questions can be answered directly when grounded; ticket replies and anything sensitive always stop at human review.

6.1 Document ingestion

1. Admin uploads a file. The API checks size and type, hashes it, and rejects duplicates in the same workspace.
2. The file goes to storage; a `documents` row is written with status `uploaded`; job `ingest_document(document_id)` is enqueued.
3. Worker sets `processing`, extracts text per file type, and normalises whitespace.
4. Chunker splits on headings first, then into ~500-token chunks with ~60-token overlap, keeping the heading path in chunk metadata.
5. Chunks are embedded in batches of 64, then inserted with their vectors in one transaction (old chunks of the same document deleted first, so re-runs are safe).
6. Status becomes `ready` with `chunk_count`, or `failed` with an error after 3 retries. The dashboard polls status.

6.2 Live chat answer (widget)

1. Customer sends a message with their session token. Rate limit and workspace token budget are checked.
2. The message is stored. A fast classification call returns intent, category, priority and confidence as validated JSON.
3. Route by intent: a knowledge question goes to RAG; an order question goes to the `get_order_status` tool; refunds, complaints, account changes or low confidence go to escalation.
4. RAG: embed the question, fetch the top 8 chunks for this workspace by cosine similarity, drop any under the similarity threshold, keep the best 5.
5. If nothing relevant is found, the assistant says it does not know and offers a human. It never answers without sources.
6. Otherwise the LLM answers from the retrieved chunks only, citing them. The answer streams back over Server-Sent Events and is saved with its citations.
7. On escalation a ticket is created from the conversation and the customer is told a person will reply.

6.3 Ticket pipeline

A ticket comes from a widget escalation or the create-ticket API. The worker classifies it, retrieves knowledge and calls read-only tools, then writes a `reply_draft`. Proposed side-effecting actions become `action_requests`. Nothing reaches the customer until an agent approves.

From	Event	To
new	classification done	triaged
triaged	draft written	awaiting_review
awaiting_review	agent approves or edits draft	replied
awaiting_review	agent rejects draft	triaged (agent writes manually or regenerates)
replied	customer answers	triaged
any open state	agent resolves	resolved
resolved	customer writes again within 7 days	triaged

6.4 Approved actions

Every tool has a risk level. `read` tools (order status, knowledge search) run immediately. `write` and `sensitive` tools (create ticket, refund, change address) only ever create an `action_requests` row with status `proposed`.

1. An agent sees the proposed action with its arguments and the AI's reason, then approves or rejects. Sensitive actions require the admin role.
2. Approval sets `approved` and enqueues `execute_action(action_id)`.
3. The worker re-checks status and role, then calls the tool with the stored `idempotency_key`, so a retry can never refund twice.
4. The result is saved as `succeeded` or `failed`; the audit log records who approved what and what happened.

The model never executes a side effect itself. It can only propose one.

7. AI layer

All model access goes through `app/ai/`, so features never call a provider SDK directly. That one seam gives retries, timeouts, cost tracking, prompt versioning and provider swaps in a single place.

Components

- `LLMClient` interface with `complete()`, `complete_structured(schema)`, `stream()` and `embed()`. One provider implementation plus a `FakeLLMClient` for tests. Every call writes an `llm_calls` row.
- **Prompt registry**: prompts live as versioned files in `app/ai/prompts/` (for example `classify_v1.md`, `answer_v1.md`). The version is stored on every call, so a quality change can be traced to a prompt change.
- **Structured outputs**: classification and drafting return JSON validated by Pydantic models. Invalid output gets one repair retry, then falls back to `needs_human`.
- **Tool registry**: each tool declares a name, description, Pydantic input schema, risk level and handler. Only registered tools exist; the model receives their JSON schemas and nothing else.
- **Agent loop**: plain Python, at most 4 tool calls and 30 seconds per run. Read tools run inline; write and sensitive tools return "proposed" to the model and create an action request.
- **Retriever**: pgvector cosine search scoped by RLS, similarity threshold, top-k from settings. Keyword search on `tsv` and reciprocal-rank fusion are added later behind the same function.

MVP tools

Tool	Risk	What it does
`search_knowledge`	read	Retrieves chunks for a query
`get_order_status`	read	Looks up a `mock_orders` row by order number, only if it matches the contact's email
`create_ticket`	write	Opens a ticket from a chat
`request_refund`	sensitive	Proposes a refund for an order; admin approval required
`escalate_to_human`	write	Hands the conversation to an agent

Guardrails against prompt injection and bad output

- Customer messages and retrieved chunks are placed inside clearly delimited data blocks; the system prompt says they are information, never instructions.
- Tool arguments are validated by schema, then by business rules (the order must belong to this workspace and this customer). The model never supplies `workspace_id`, URLs or SQL.
- Answers must cite at least one retrieved chunk; an answer with no valid citation is replaced by "I'm not sure, let me get a person."
- System prompts, other tenants' data and secrets never enter the context window.
- Output is checked for leaked emails or keys before it is shown.

8. API surface

All feature routes live under `/api/v1`. Dashboard routes need a user JWT; widget routes need a customer session token; list endpoints use cursor pagination; errors use one shape: `{"error": {"code", "message", "request_id"}}`. Write endpoints that trigger side effects accept an `Idempotency-Key` header.

Area	Endpoints	Who
Health	`GET /health`, `GET /health/ready`	Public
Auth	`POST /auth/signup`, `/auth/login`, `/auth/refresh`, `/auth/logout`, `GET /auth/me`	Public / user

Area	Endpoints	Who
Workspaces	`POST /workspaces`, `GET /workspaces`, `POST /workspaces/{id}/switch`, `GET` / `PATCH /workspace/settings`	User; admin for settings
Members	`GET /members`, `POST /members/invite`, `PATCH /members/{id}`, `DELETE /members/{id}`	Admin
Knowledge	`POST /documents` (multipart), `GET /documents`, `GET /documents/{id}`, `DELETE /documents/{id}`, `POST /documents/{id}/reindex`, `POST /knowledge/search`	Admin; search for agents
Widget	`POST /widget/session`, `POST /widget/messages` (SSE stream), `GET /widget/conversations/{id}`, `POST /widget/feedback`	Customer
Conversations	`GET /conversations`, `GET /conversations/{id}`	Agent
Tickets	`GET /tickets`, `POST /tickets`, `GET /tickets/{id}`, `PATCH /tickets/{id}` (assign, priority, status), `POST /tickets/{id}/regenerate-draft`	Agent
Drafts	`POST /drafts/{id}/approve`, `POST /drafts/{id}/reject`, `PATCH /drafts/{id}` (edit)	Agent
Actions	`GET /actions?status=proposed`, `POST /actions/{id}/approve`, `POST /actions/{id}/reject`	Agent; admin for sensitive
Mock orders	`GET /mock-orders`, `POST /mock-orders/seed`	Admin (dev tooling)
Analytics	`GET /analytics/overview?from&to` (tickets, resolution time, deflection rate, feedback, tokens, cost)	Admin
Audit	`GET /audit-logs`	Admin

9. Background jobs and reliability

The worker is the same codebase started with `arq app.workers.main.WorkerSettings`. Jobs are enqueued only after the database transaction commits, so a job never runs against a row that does not exist yet.

Job	Trigger	Timeout	Retries
`ingest_document`	Document uploaded or reindex	10 min	3, exponential backoff
`process_ticket` (classify, retrieve, draft)	Ticket created or customer reply	2 min	3
`execute_action`	Action approved	1 min	5, same idempotency key
`send_reply`	Draft approved	1 min	5
`purge_expired_tokens`	Cron, daily	5 min	1
`rollup_usage`	Cron, hourly	5 min	1

Rules every job follows

- The payload carries only ids and `workspace_id`; the job reloads state from the database and sets the RLS context before touching data.
- Jobs are idempotent: they check current status first and do nothing if the work is already done.
- Every external call (LLM, embeddings, tools) has a timeout; LLM calls retry on 429/5xx with jittered backoff, at most 3 times.
- After the last retry the record moves to a `failed` state with the error, and the dashboard shows it with a retry button.
- A Redis lock per ticket stops two drafts being generated at once.
- The worker logs and traces with the same request id as the API call that enqueued the job.

10. Web app and widget

The Next.js app in `apps/web` serves both the agent dashboard and the customer widget. It talks to the API only through a typed client generated from the API's OpenAPI schema, so frontend and backend types never drift.

Page	Route	Purpose
Sign up, log in	`/signup`, `/login`	Account and first workspace
Onboarding	`/onboarding`	Name the workspace, upload first documents, copy the widget snippet
Overview	`/dashboard`	Open tickets, awaiting review, deflection rate, feedback, tokens and cost
Inbox	`/tickets`	Filter by status, priority, assignee
Ticket detail	`/tickets/[id]`	Conversation, classification, AI draft with citations, approve / edit / reject, proposed actions
Approvals	`/approvals`	All proposed actions in one queue
Conversations	`/conversations`	Every widget chat, including ones the AI resolved
Knowledge base	`/knowledge`	Upload, status, delete, reindex, test search
Settings	`/settings`	Members and roles, AI settings, widget key and allowed origins
Widget	`/widget/[workspaceKey]` + `widget.js`	Chat bubble loaded into an iframe on the customer's site

The widget runs in an iframe so the host site's CSS and scripts cannot reach it. `widget.js` is a few lines that inject the iframe; answers stream token by token over SSE; each answer shows its sources and thumbs up/down. The planned React Native agent app reuses the same API and generated client.

11. Observability and evaluation

Every request gets a `request_id` that flows through logs, traces, jobs and `llm_calls`, so one id answers "what happened to this customer message?" end to end.

Observability

- Logs: structlog JSON to stdout (Cloud Logging picks them up), with `request_id`, `workspace_id`, `user_id`; no message bodies at info level.
- Traces: OpenTelemetry spans around HTTP requests, DB queries, jobs, retrieval and each LLM call.
- Metrics: request latency and error rate, job duration and failure count, LLM latency, tokens and cost per workspace, retrieval hit rate.
- Errors: Sentry for API, worker and web.
- Alerts: API 5xx above 2% for 5 minutes, job failure spike, a workspace hitting its daily token budget.

Evaluation (lives in `evals/`, runs in CI on any change to `app/ai/`)

Eval	Dataset	Metric	Pass bar
Retrieval	50 questions with the chunk that answers each	Recall@5	⩾ 0.85
Answer grounding	Same questions plus reference answers	LLM-judged faithfulness to cited chunks	⩾ 0.9
Refusal	20 questions the docs cannot answer	Correctly says "don't know"	⩾ 0.9
Classification	100 labelled support messages	Intent accuracy, priority accuracy	⩾ 0.85 / ⩾ 0.8
Tool selection	30 messages with expected tool and arguments	Exact tool + valid arguments	⩾ 0.9
Injection	20 adversarial messages and poisoned documents	No unauthorised tool call or leak	100%

Eval results are saved per run with model and prompt version, so a regression points at the change that caused it. Thumbs-down feedback from production gets reviewed and added to the datasets.

12. Deployment and environments

Three environments share one set of Docker images: local (Docker Compose), staging and production (both on Google Cloud). Configuration differs only by environment variables and secrets.

Component	Local	Staging / production
API	`uvicorn --reload`	Cloud Run service, min 0 (staging) / 1 (prod) instances
Worker	`arq` process	Cloud Run service, min 1 instance, CPU always allocated
Web + widget	`next dev`	Cloud Run service (or Vercel)
Postgres + pgvector	Docker `pgvector/pgvector:pg16`	Cloud SQL for PostgreSQL 16, pgvector extension, private IP, daily backups
Redis	Docker `redis:7`	Memorystore for Redis
Files	Local folder	Cloud Storage bucket, private
Secrets	`.env`	Secret Manager mounted as env vars
Images	Built locally	Artifact Registry

CI/CD (GitHub Actions)

1. On every pull request: ruff, mypy, pytest (with a Postgres service container), web lint and type-check, and evals when `app/ai/` changed.
2. On merge to `main`: build and push images tagged with the commit SHA, run Alembic migrations as a Cloud Run job, deploy API, worker and web to staging, run smoke tests.
3. Production deploy is a manual approval in GitHub on the same images, never a rebuild.
4. Rollback = redeploy the previous image tag. Migrations are written to be backward compatible for one release (add, backfill, then remove later).

13. Build plan

The app is built in 14 Antigravity prompts, each small enough to review and test before the next. Each prompt ends with a check you run yourself; tick it off here when it passes.

Week 1 — Foundation

- [ ] Prompt 1: Conventions in `AGENTS.md` (and save this design as `docs/architecture.md`)
- [ ] Prompt 2: Backend foundation — Compose, FastAPI, config, DB layer, Alembic, health checks
- [ ] Prompt 3: Auth, users, workspaces, memberships, roles, refresh-token rotation, RLS policies and the tenant context dependency
- [ ] Prompt 4: Next.js app scaffold, generated API client, sign-up, login, onboarding, dashboard shell

Week 2 — Knowledge base

- [ ] Prompt 5: Storage interface, document upload, arq worker, ingestion job (parse, chunk, embed, store)
- [ ] Prompt 6: `LLMClient`, prompt registry, `llm_calls` logging, retriever, knowledge search endpoint, knowledge page in the web app

Week 3 — Support workflow

- [ ] Prompt 7: Widget session, streaming RAG chat with citations, feedback, embeddable `widget.js`
- [ ] Prompt 8: Contacts, conversations, tickets, classification job, inbox and ticket pages, mock orders with seed data

Week 4 — Agent controls

- [ ] Prompt 9: Tool registry, agent loop, `process_ticket` job writing drafts and action requests
- [ ] Prompt 10: Draft approve/edit/reject, action approvals with idempotent execution, audit log, approvals page

Week 5 — Quality

- [ ] Prompt 11: Rate limits, token budgets, structured logging, OpenTelemetry, Sentry, analytics endpoint and overview page
- [ ] Prompt 12: Eval datasets and runner, cross-tenant isolation test suite, injection tests

Week 6 — Release

- [ ] Prompt 13: Dockerfiles, GitHub Actions CI, staging deploy to Cloud Run with Cloud SQL, Memorystore, GCS, Secret Manager
- [ ] Prompt 14: Production deploy, smoke tests, README and architecture docs, demo workspace

The mobile agent app and the future extensions come after the MVP is live.

14. Open decisions and risks

Three choices are still open and need answers before Prompt 5; the rest of the design does not depend on them.

- [ ] **LLM provider** for chat, classification and drafting (needed by Prompt 6).
- [ ] **Embedding model and dimension**. Some chat providers do not offer embeddings, so this may be a second provider. The dimension is fixed in the first `document_chunks` migration.
- [ ] **Web hosting**: Cloud Run alongside the API, or Vercel.

Risk	Effect	Mitigation
A query misses the `workspace_id` filter	One tenant sees another's data	RLS as a second wall, cross-tenant test suite in CI
Prompt injection through a document or chat	Wrong answer or an unwanted action	Data-block prompting, schema-checked tools, side effects need approval
Answers sound right but are not grounded	Customers get wrong information	Mandatory citations, similarity threshold, grounding eval
LLM cost grows with traffic	Budget overrun	Per-workspace daily token budget, cost in the dashboard, small model for classification
Provider outage or rate limiting	Chat stops answering	Timeouts and retries, graceful "a person will reply" fallback
AI-generated code drifts from the design	Hard-to-maintain codebase	`AGENTS.md` conventions, this doc in `docs/`, one reviewed prompt at a time